Monte carlo tree search



Zola Mahlaza

Department of Computer Science

Feb 2026

Contents

- Monte Carlo tree search

Limitations of alpha-beta

- Alpha–beta tree search has limitations
  - In GO, b starts at 361. Alpha beta pruning can yield 19 (with perfect move ordering)
  - For a depth of 8, ~sixteen billion states!
  - Most positions are in flux

    until the endgame
  - Material value is not a strong indicator
- Monte Carlo tree search
  - simulation over heuristic function



Image src: Wikipedia

Monte carlo tree search

- Use framework to understanding approach
  - Consider a single state each
  - There are multiple (finite) actions that can be taken
  - For each chosen action, there is a ‘reward’
  - Goal: Maximize the expected total ‘reward’ over ‘time’ period
- Do not calculate every possibility. Determine move that survives the most possible futures

Monte carlo tree search

- Use framework to understanding approach
  - Consider a single state each
  - There are multiple (finite) actions that can be taken
  - For each chosen action, there is a ‘reward’
  - Goal: Maximize the expected total ‘reward’ over ‘time’ period
- Do not calculate every p
  move that survives the

Purpose of simulations

Monte carlo tree search

- Value of a state is estimated
  - Average utility over a number of simulations from the state
  - Each simulation is for a complete game
  - e.g., play N games and obtain scores {s1, s2, …, Sn}
- For binary games, estimated value is win percentage

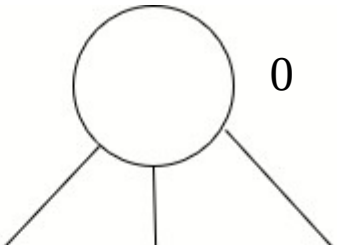
Simulation/playout

- In simulation, you make moves for both agents
  - Actions can be chosen randomly (simple games)
  - For most games, create a playout policy based on self-play
- Create an unintelligent agent that plays itself
- One instance of the agent will win the game (by chance)
- Use the game history to assign a score for each position → dataset to improve model
- Also, possible to use game-specific heuristics

Pure Monte Carlo search

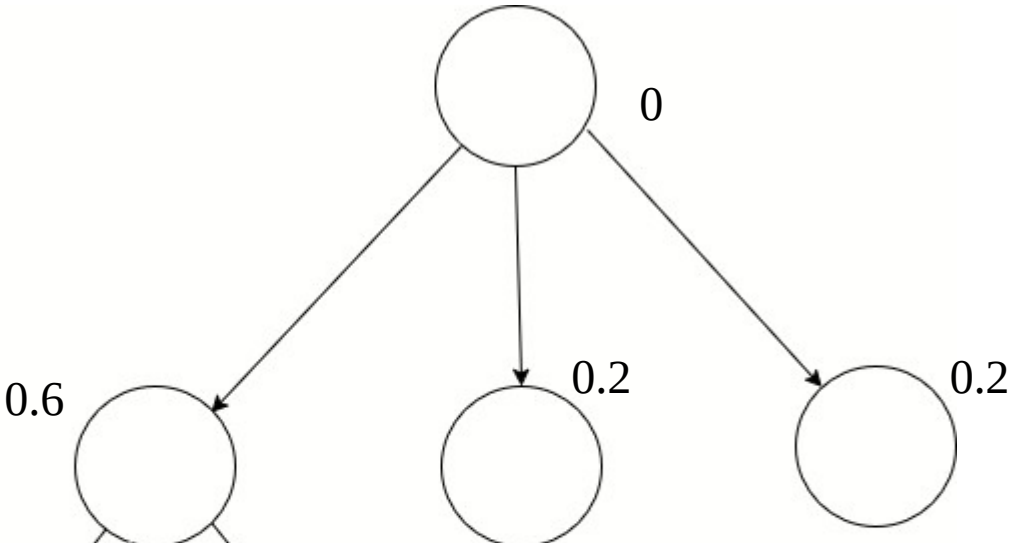
- N simulations starting from the current state of the game
- Identify moves the highest estimated percentage
- This is a slightly different algorithm from Monte Carlo Tree search

Pure Monte Carlo search example

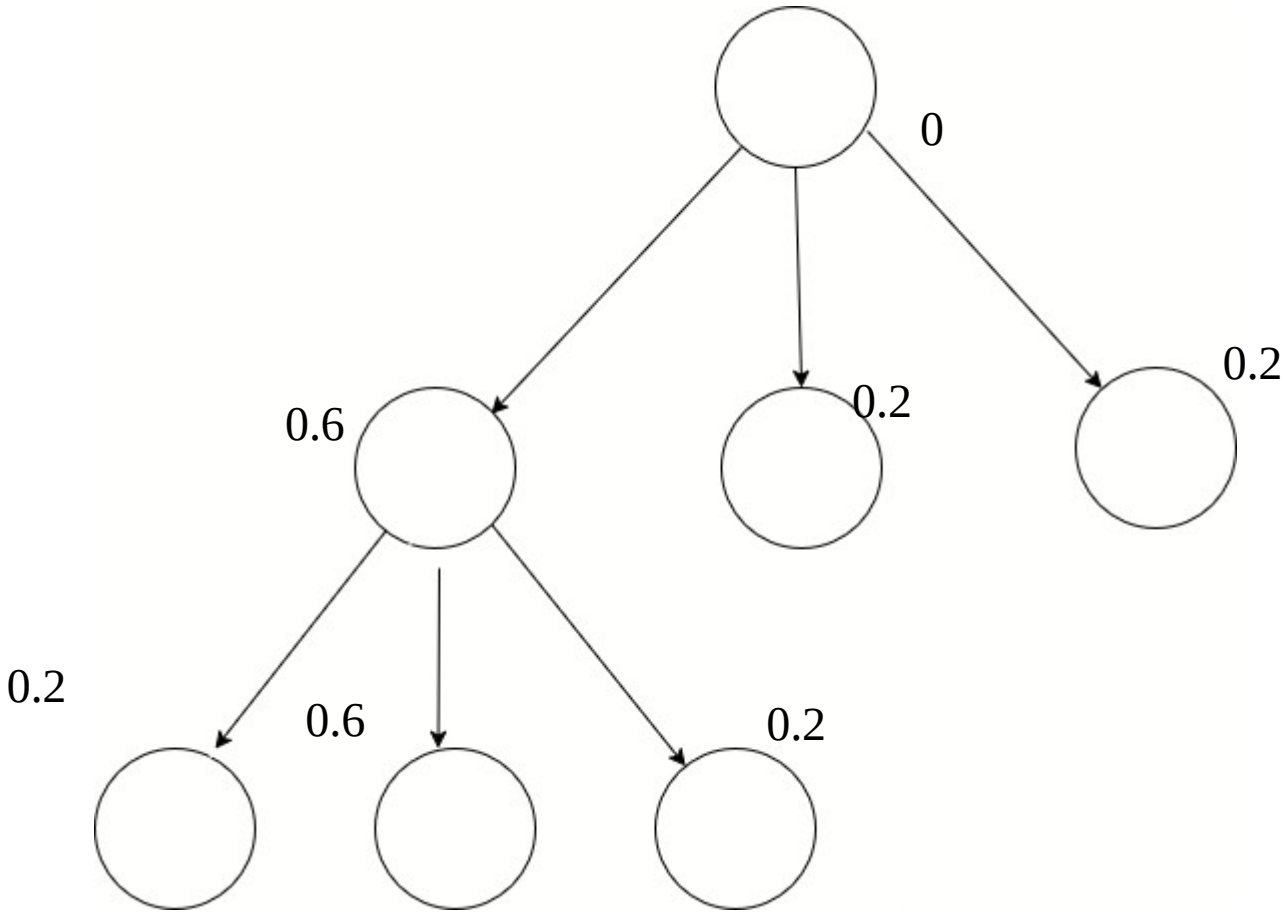


Expand via each action

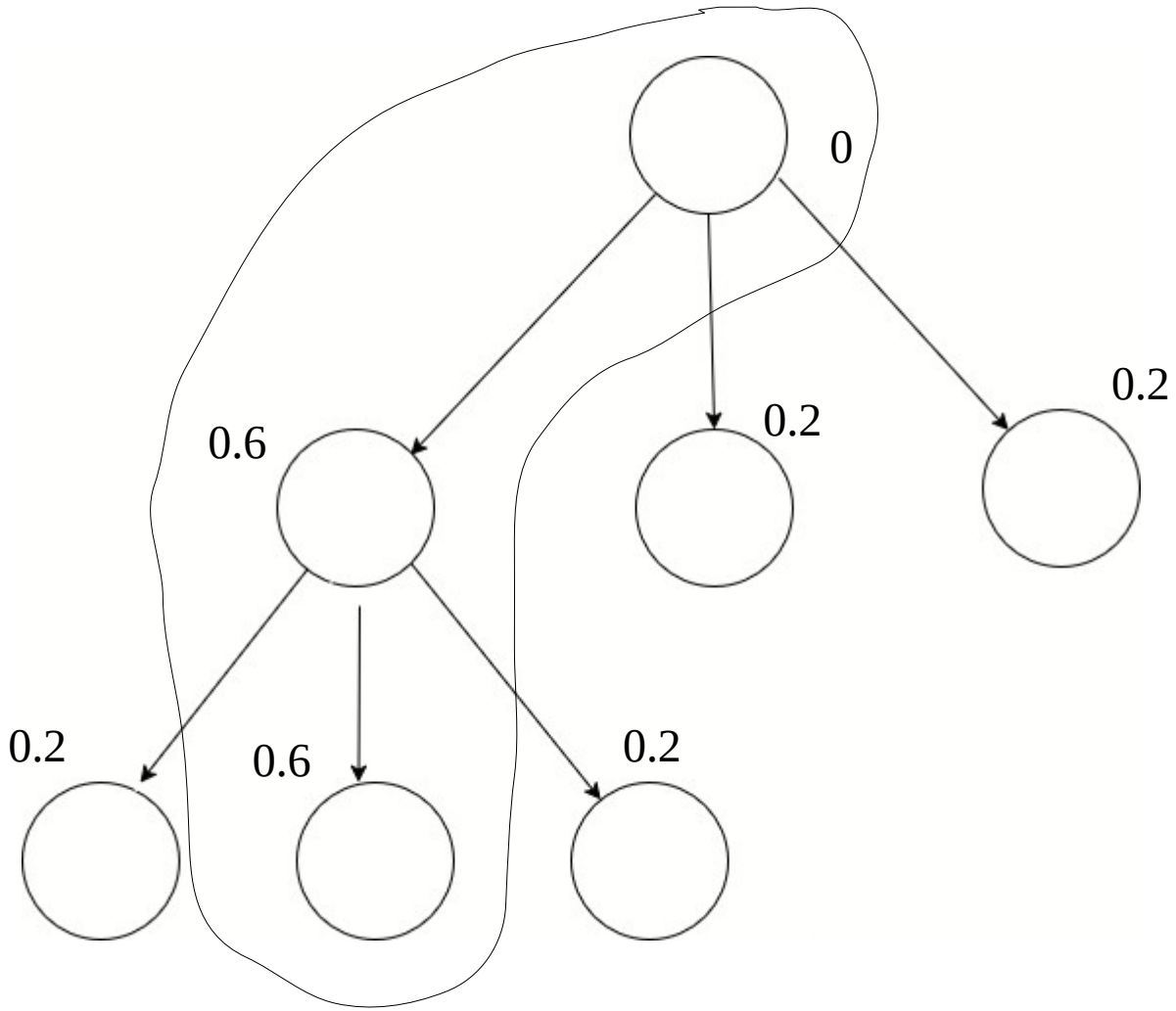
Pure Monte Carlo search



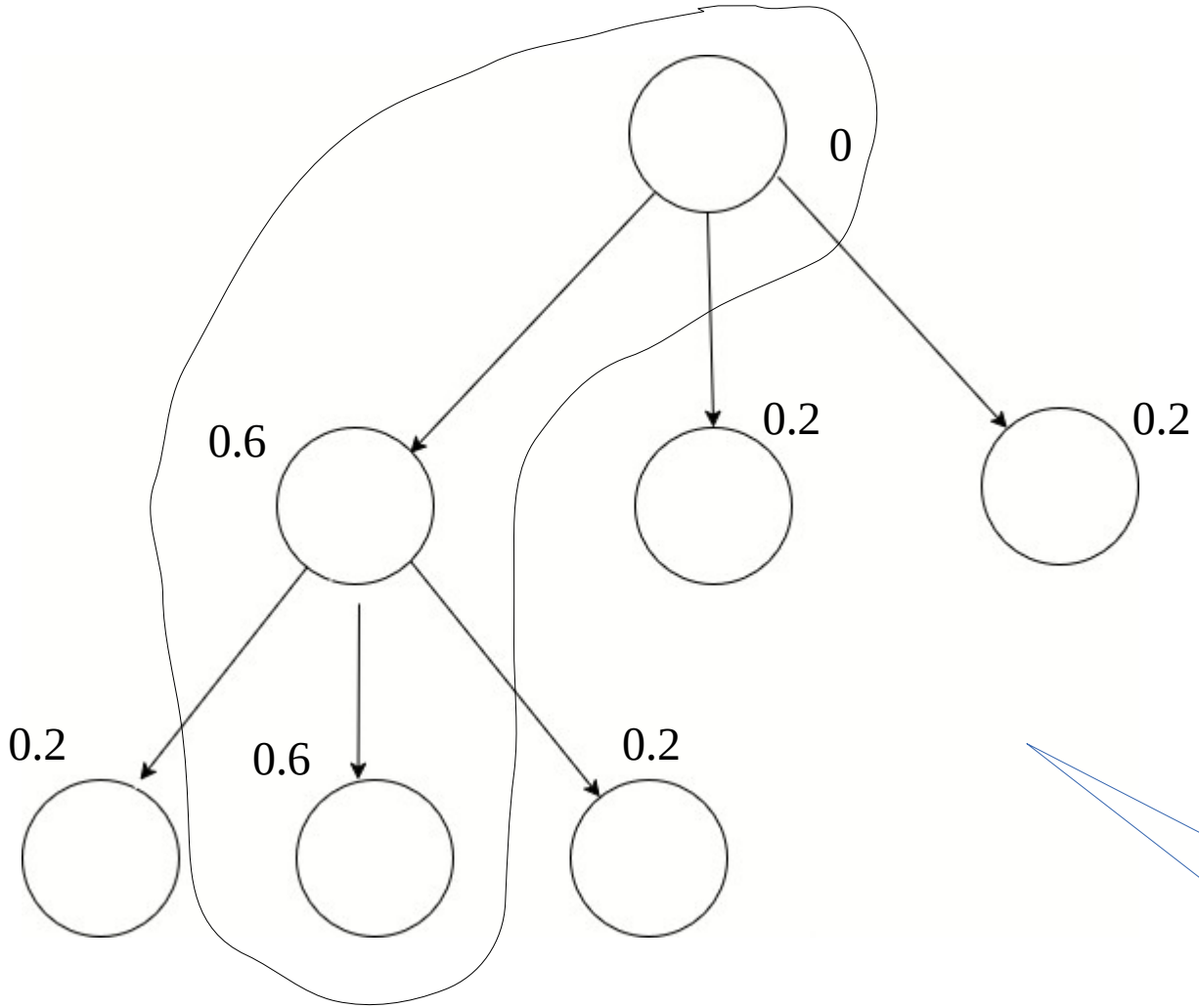
Pure Monte Carlo search



Pure Monte Carlo search



Pure Monte Carlo search



Practically,
This is flat because there is no need to keep track of the tree

Monte Carlo search

- Greedy strategy is an example of exploitation
  - Ignores tactics!
- We need to balance exploration and exploitation
  - We need a selection policy
- Example: Upper Confidence Bound 1
  - U(n) = total utility for all playout through n
  - N(n) = # of simulations at n (i.e., visits to n)
  - C = value (greater than zero) to balance exploration vs exploitation
    - e.g., greedy c = 0
    - In theory, you aim for a value around 1.41

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

Monte Carlo tree search

- Selection: We begin at root, identify successor state using selection policy. Repetitively, until terminal state
- Expansion: Expansion: create node in tree and mark parent. New node marked with utility (e.g., 0/0 for wins in 0 simulations)
- Simulation: perform playout via policy. Update node utility afterwards
- Back-propagation: update search tree nodes from current node to the root

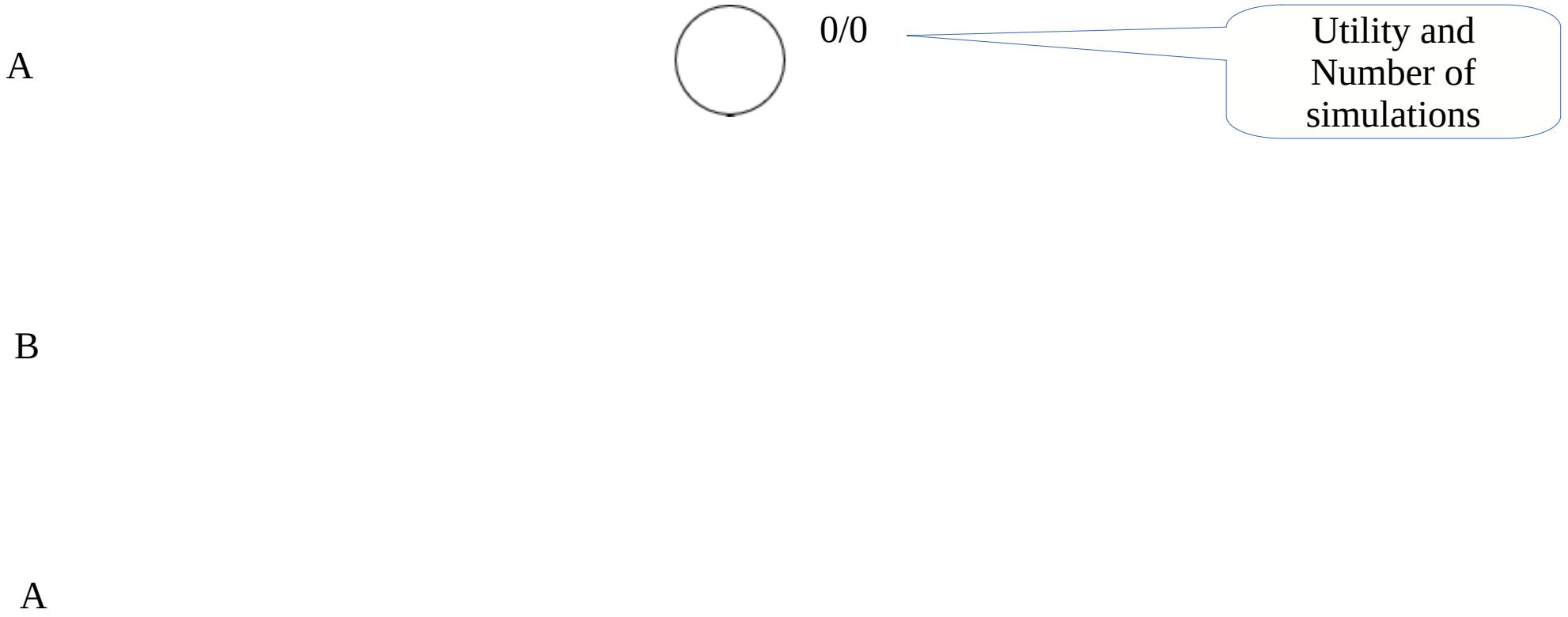
Monte Carlo tree search

```
function MONTE-CARLO-TREE-SEARCH(state) returns an action
    tree ← NODE(state)
    while IS-TIME-REMAINING() do
        leaf ← SELECT(tree)
        child ← EXPAND(leaf)
        result ← SIMULATE(child)
        BACK-PROPAGATE(result, child)
    return the move in ACTIONS(state) whose node has highest number of playouts
```

Time limit or fixed number of iterations

Monte Carlo tree search example

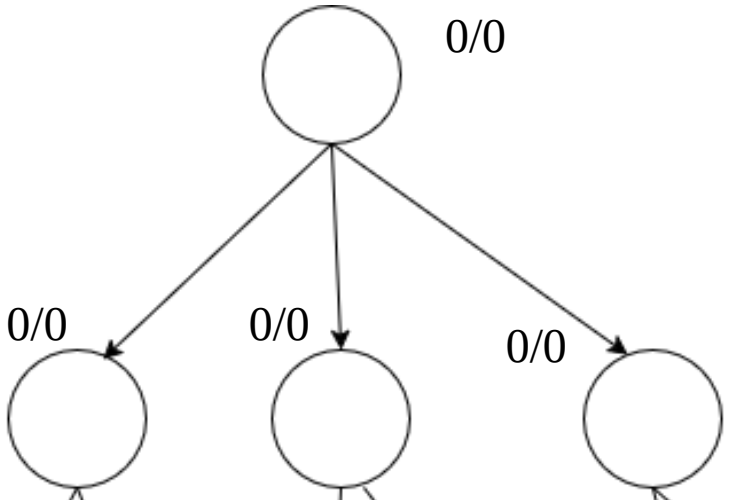
- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game

A



How to select?

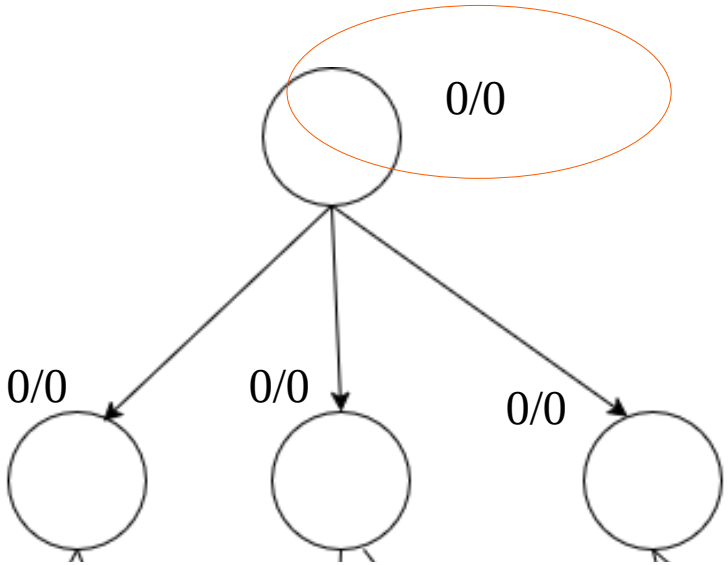
B

A

Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game

A



How to select?

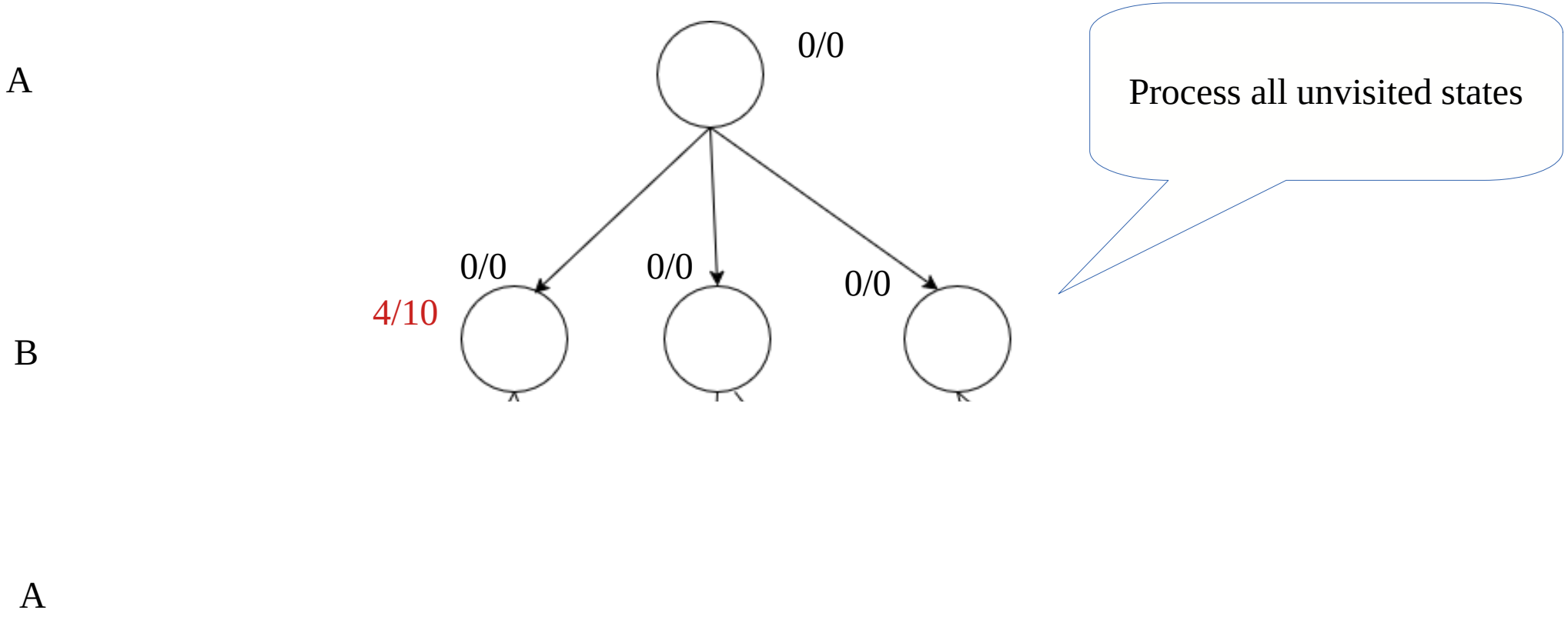
B

A

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

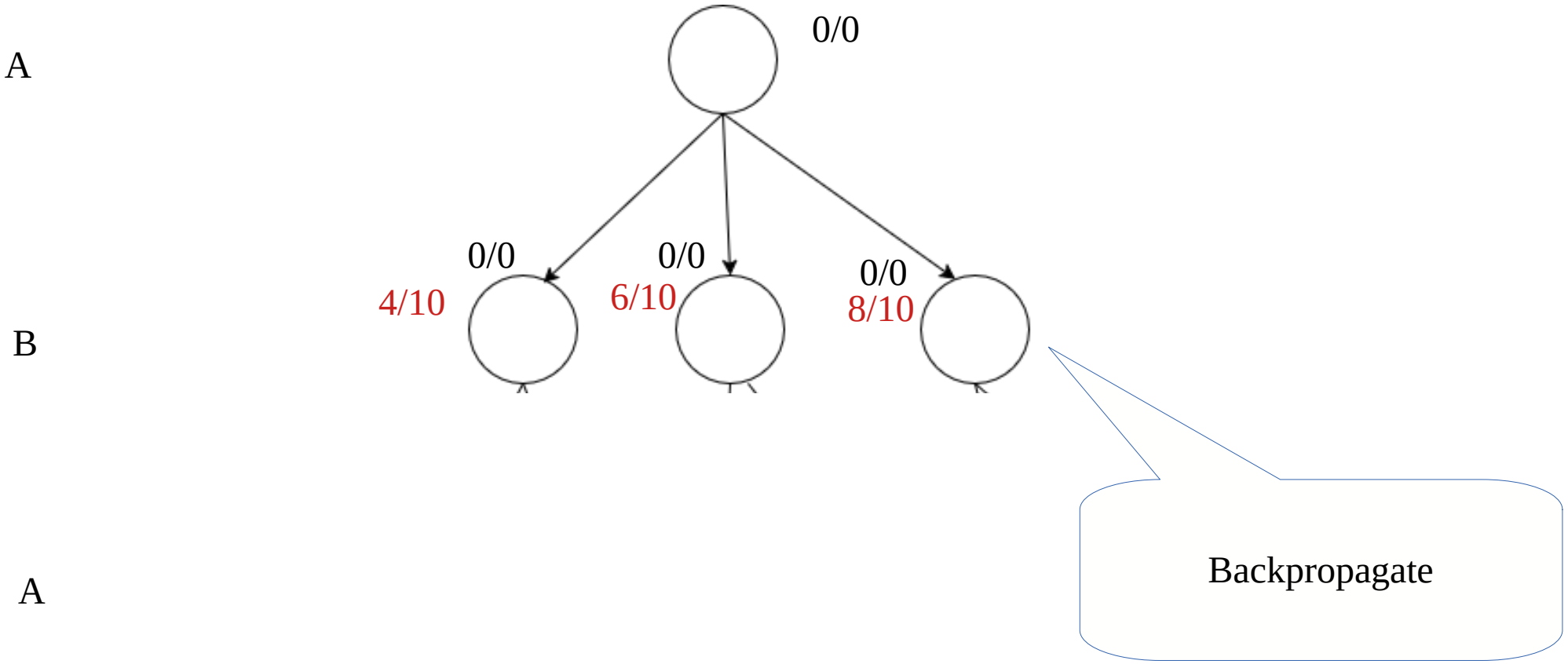
Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



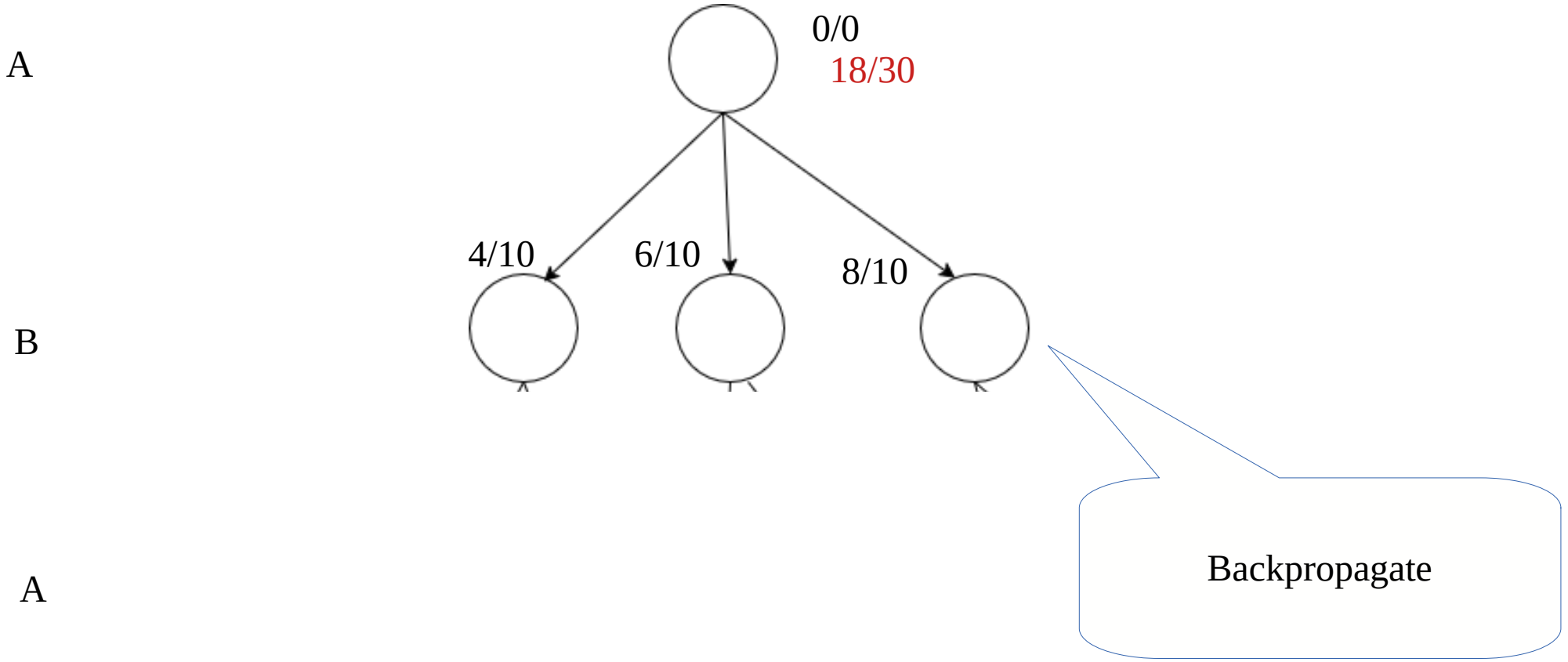
Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



Monte Carlo tree search example

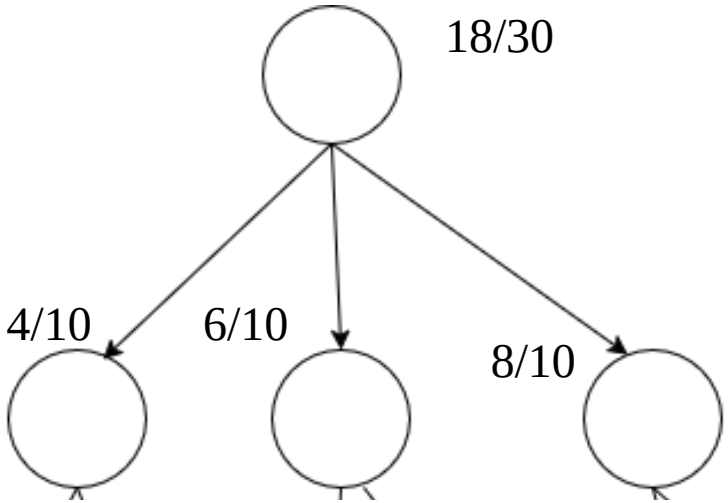
- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game

A



Select 8/10 since c = 0

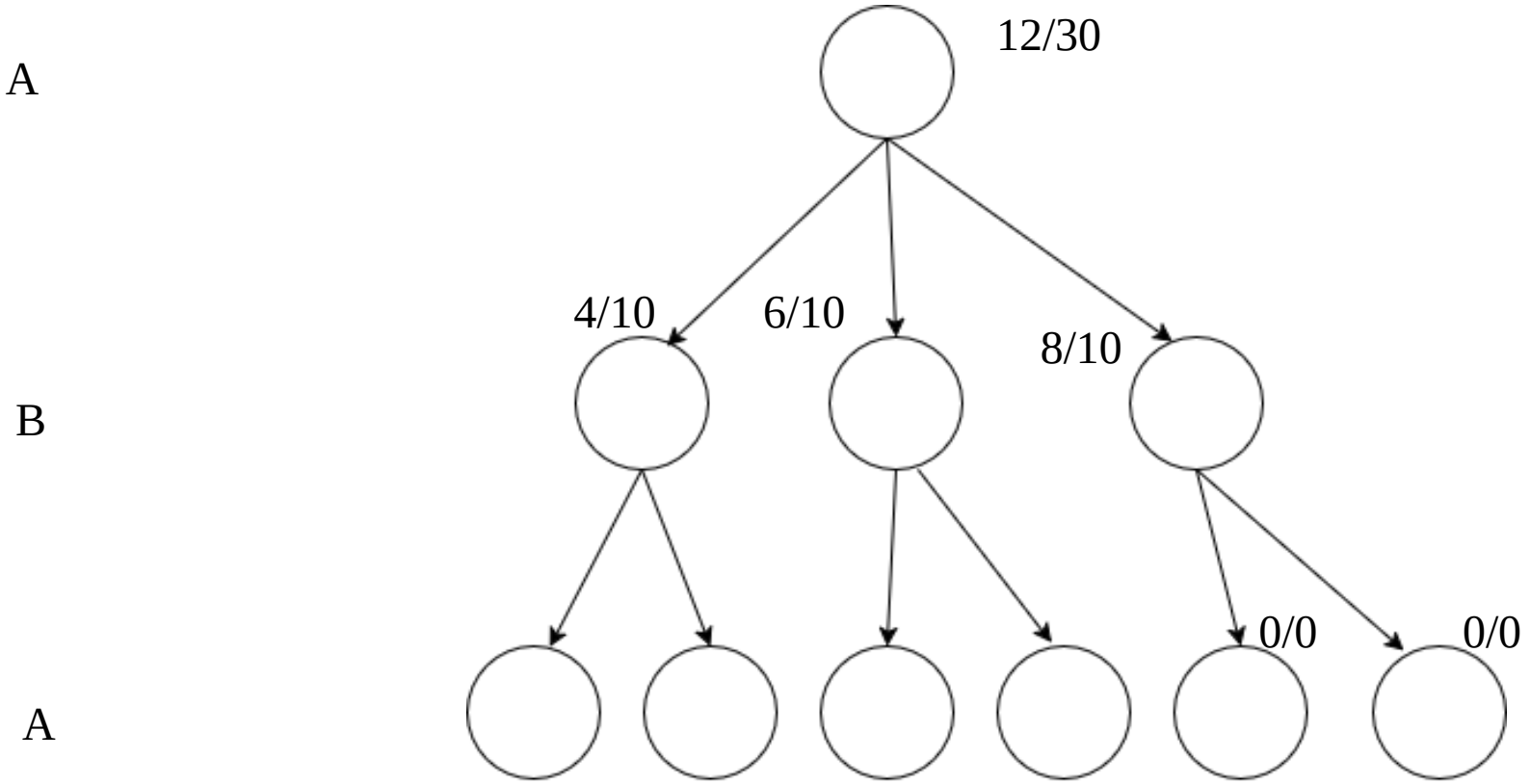
B

A

$$UCB1(n) = \frac{U(n)}{N(n)} + C \times \sqrt{\frac{\log N(\text{PARENT}(n))}{N(n)}}$$

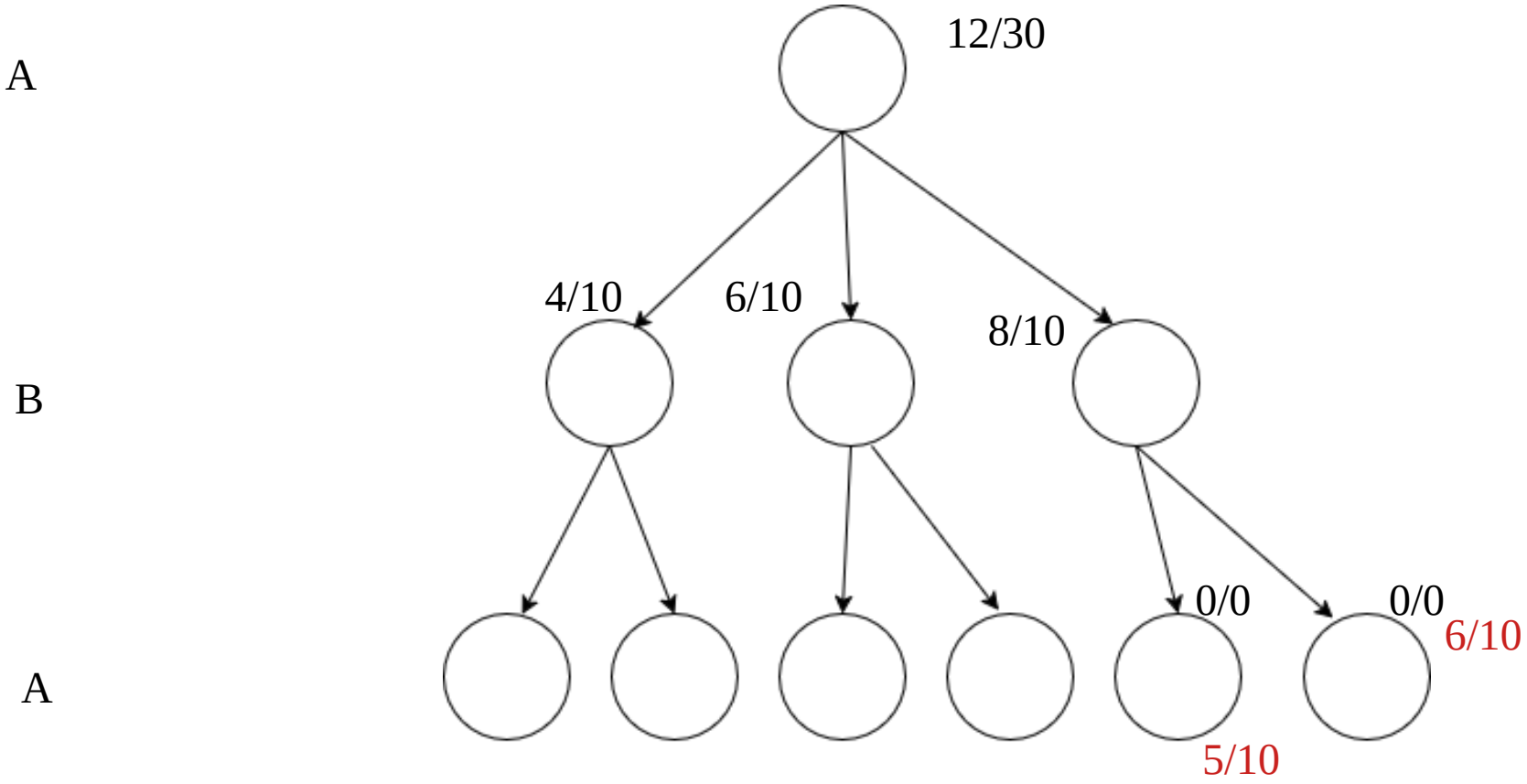
Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



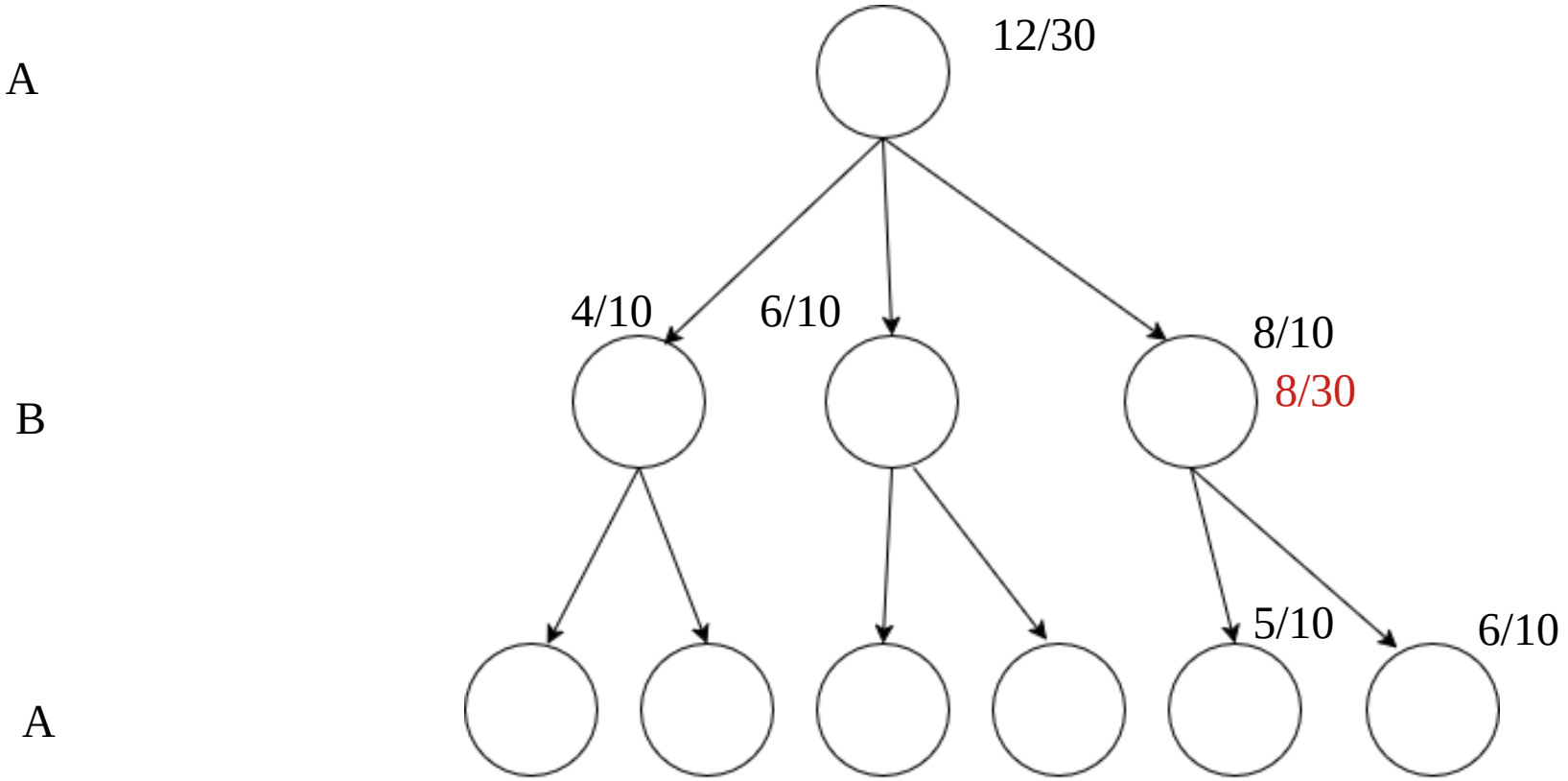
Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



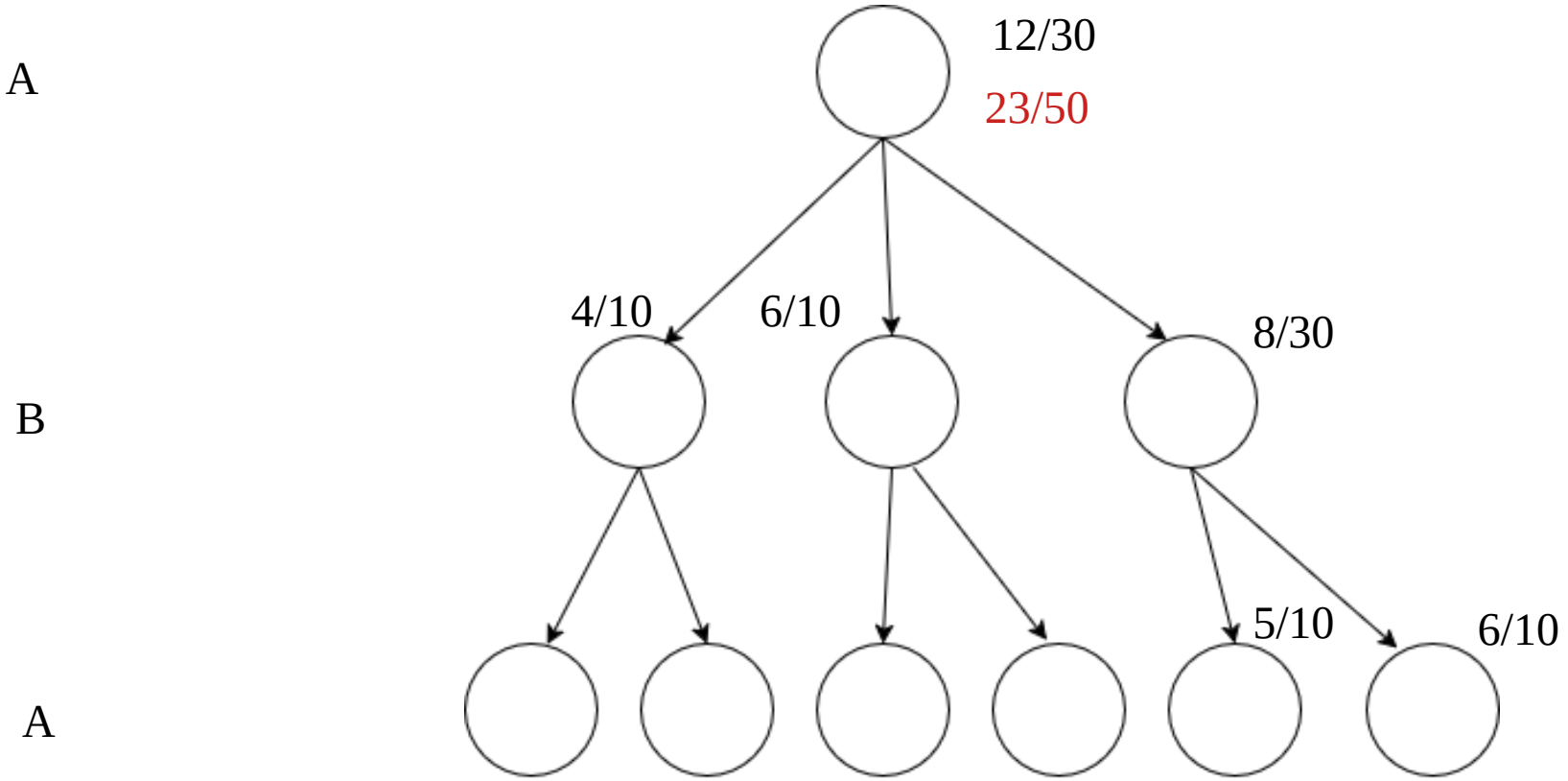
Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



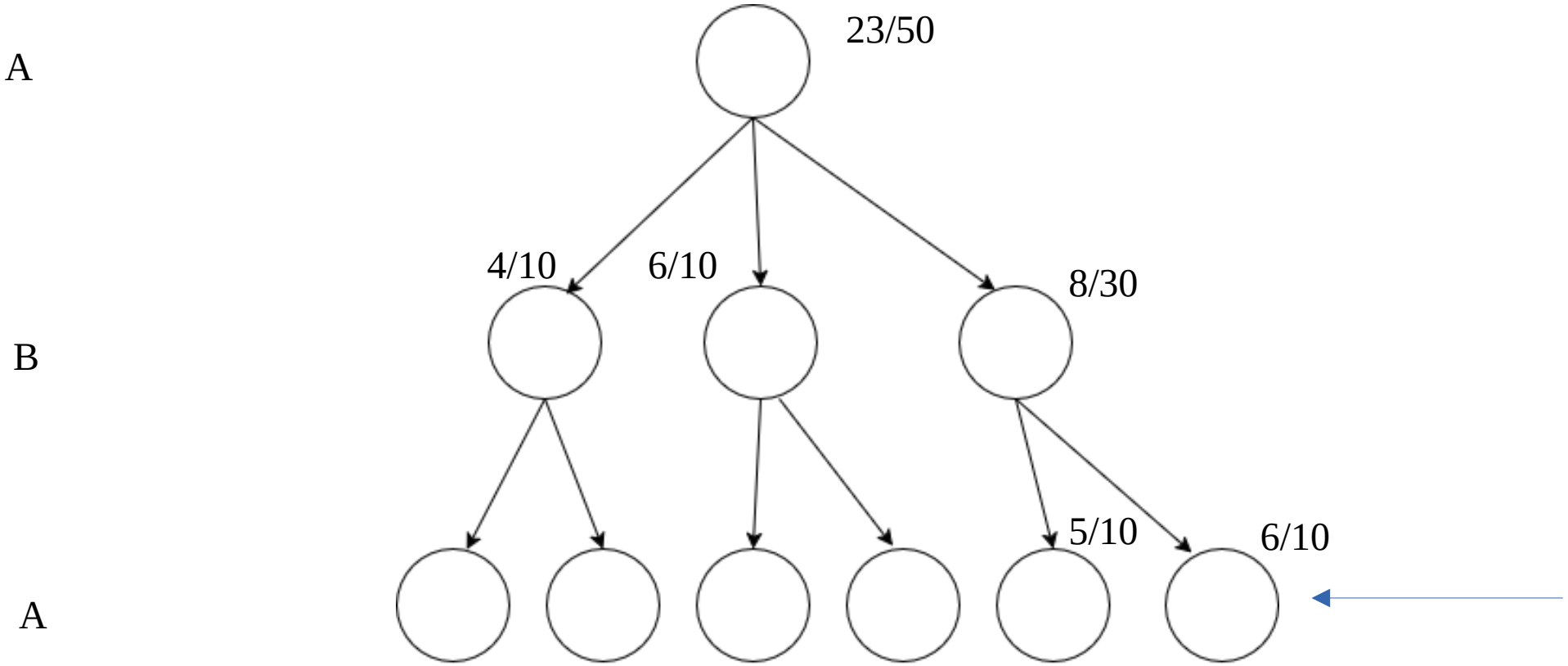
Monte Carlo tree search example

- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game

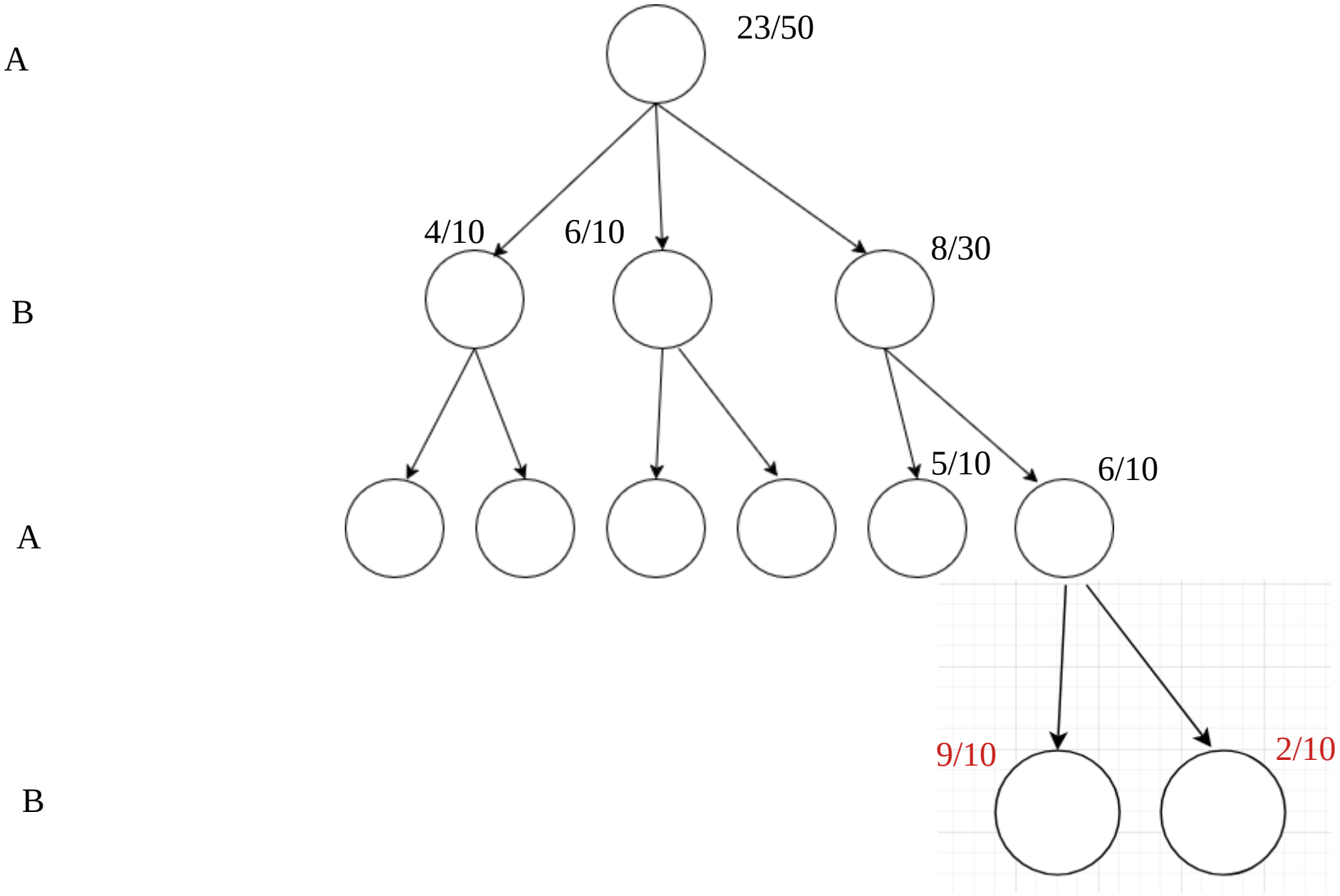


Monte Carlo tree search example

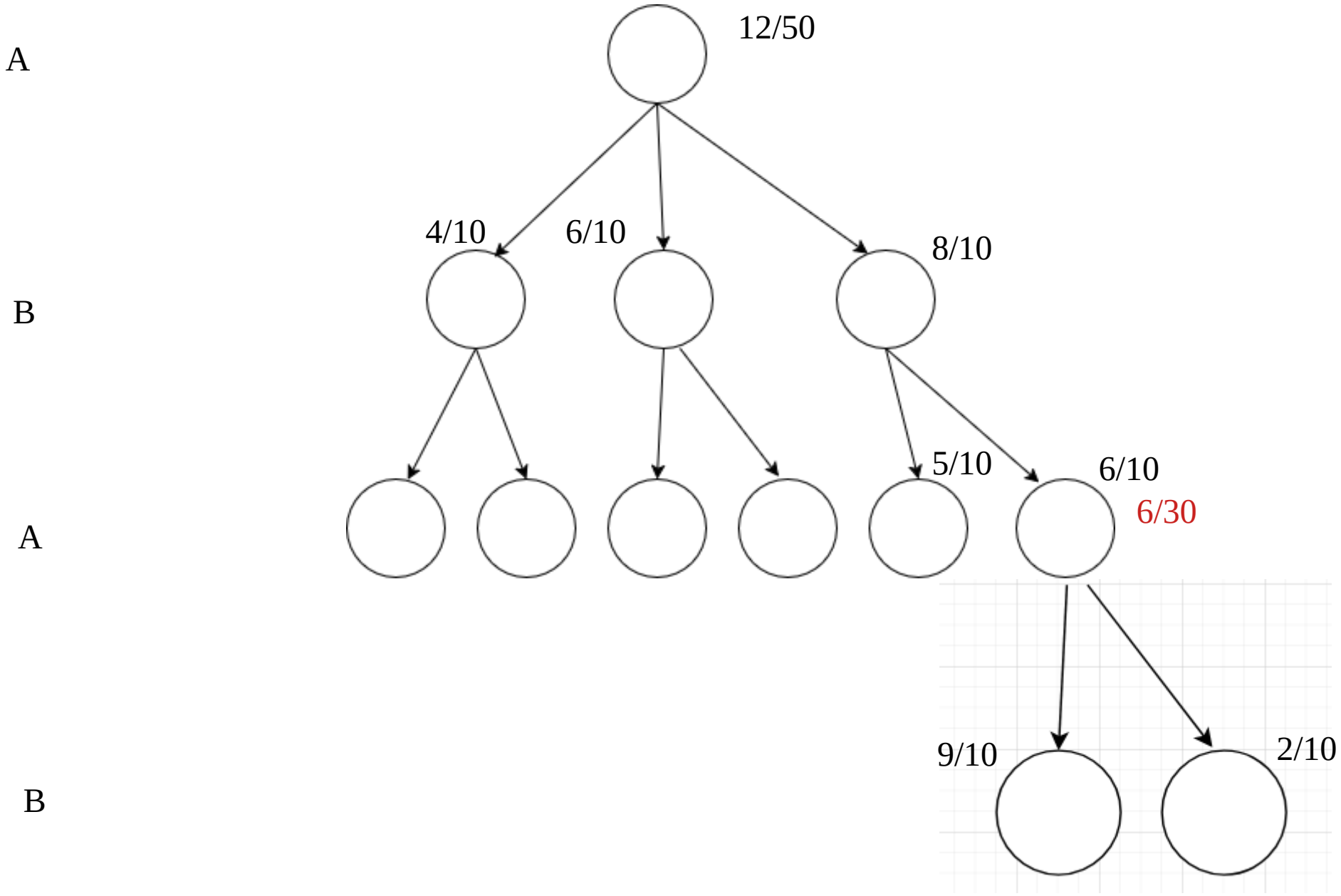
- Set c = 0 and simulations = 10
- There are two agents, A and B, in turn based game



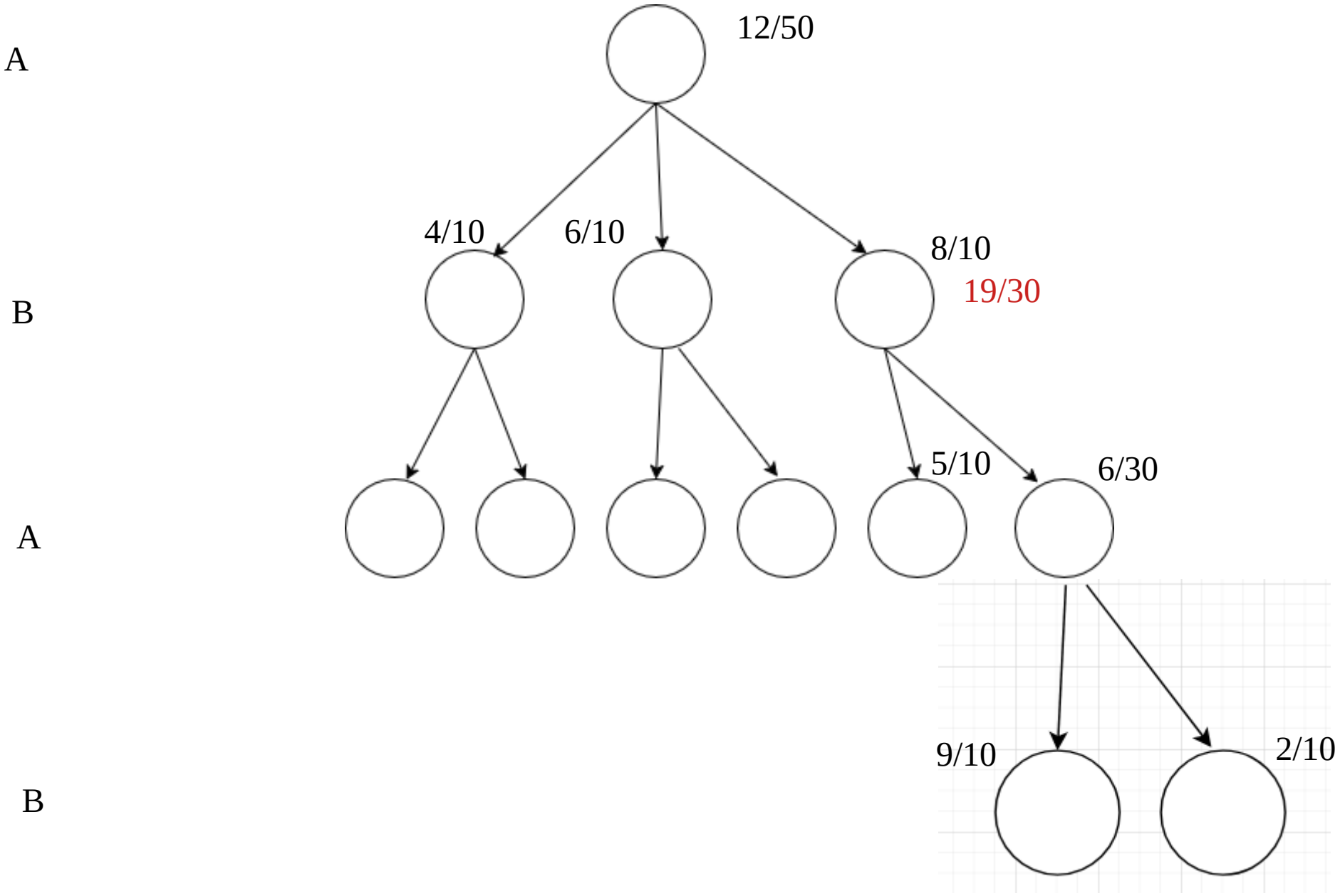
Monte Carlo tree search example



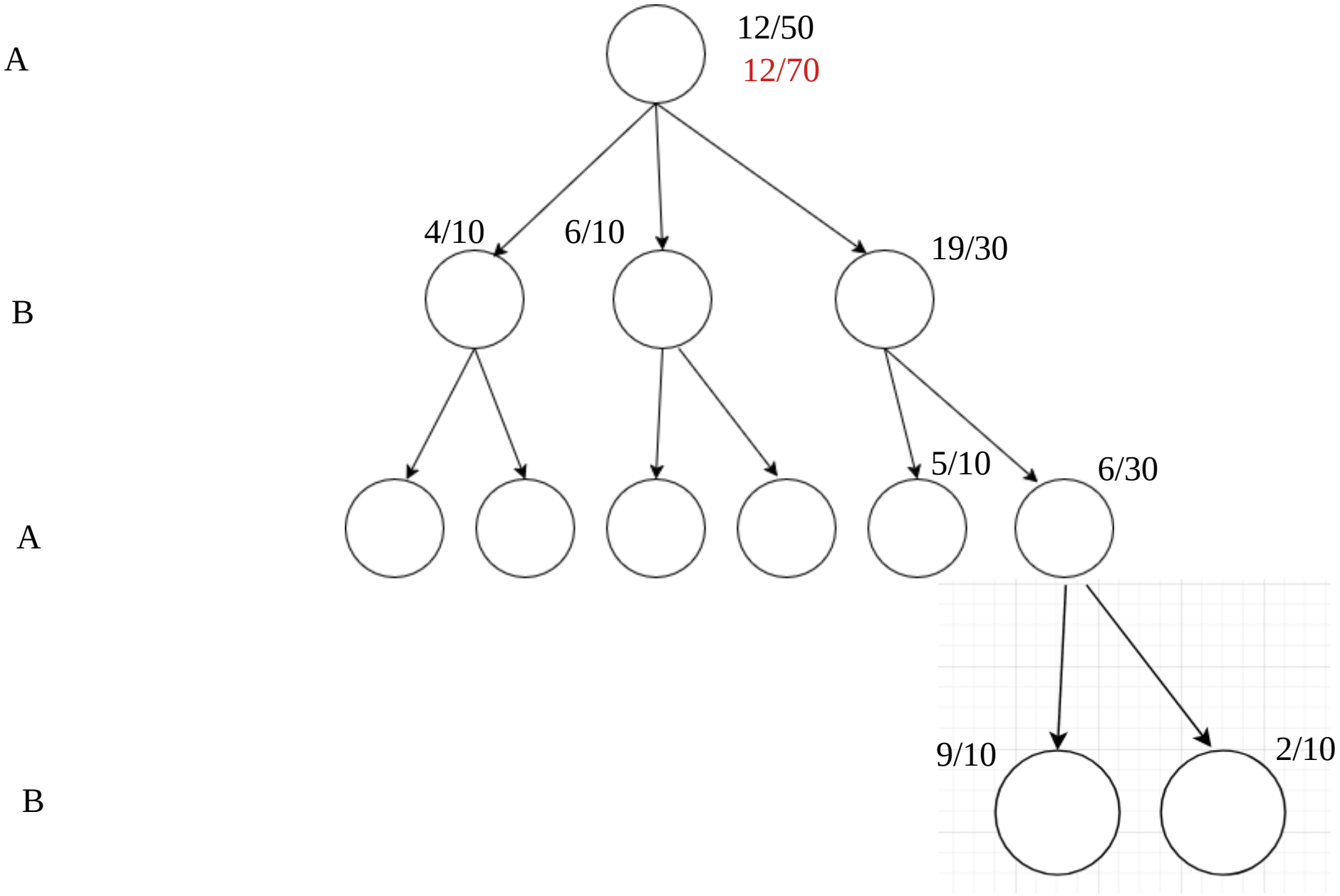
Monte Carlo tree search example



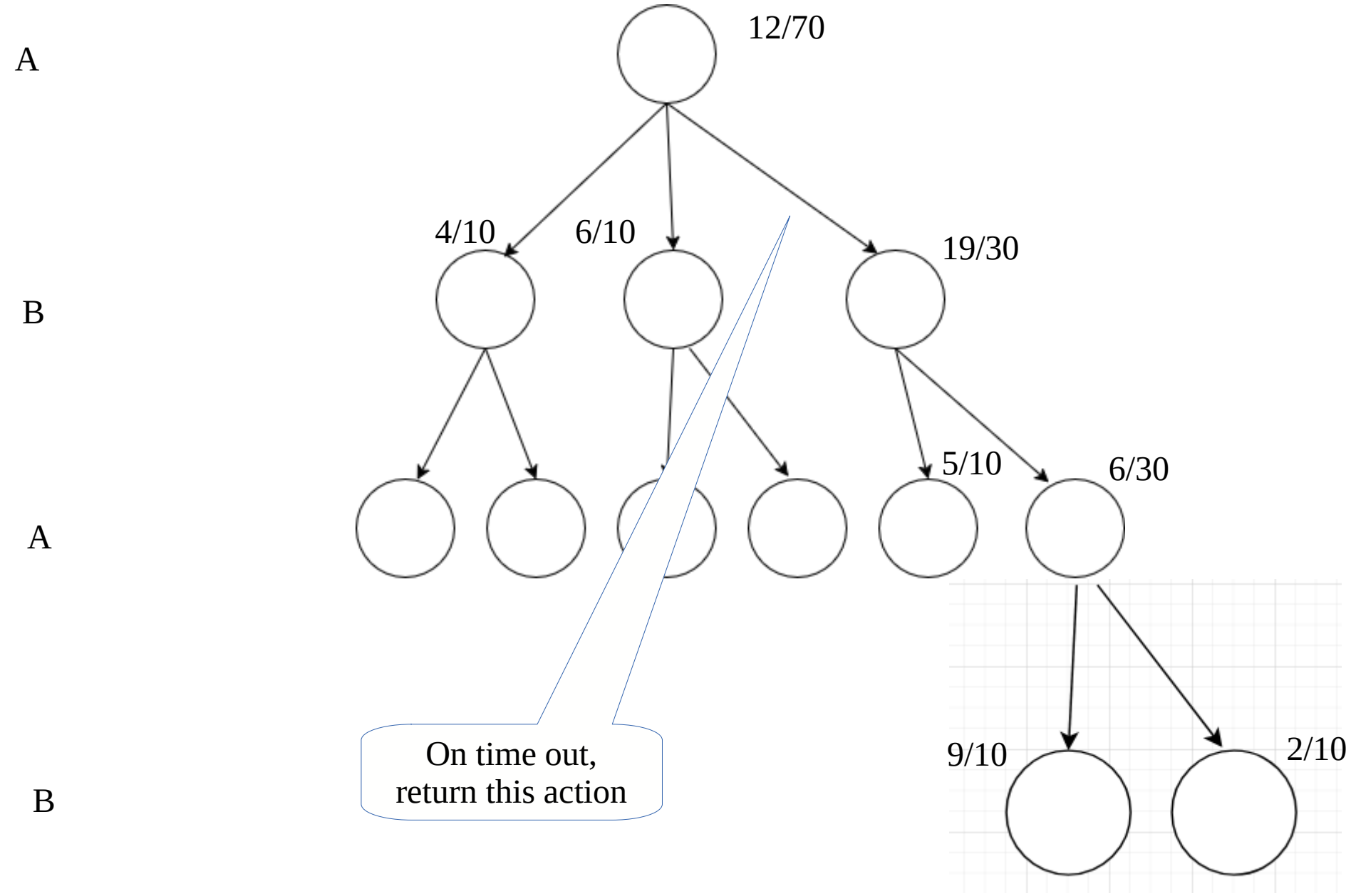
Monte Carlo tree search example



Monte Carlo tree search example



Monte Carlo tree search example



Monte Carlo tree search

- The algorithm is able to handle massive state spaces
- It is heuristic-free
- You can explore various depths of the tree per move

Reading

- Sections 5.4

